Université de Monastir
Institut Supérieur d'Informatique et de Mathématiques

Cours: Conception par Contrats

Filière: MR1-GL

Chapitre 3: Introduction à la programmation par contrat

Réalisé par:
Dr. Sakka Rouis Taoufik

I. Vérification/Validation des logiciels

- Un logiciel est incorrect (ne fait pas le travail demandé) ssi il existe un **écart** entre la **spécification** et **l'implémentaion**
- Pour résoudre l'écart entre la spécification et l'implémentation, on fait appel à la **technique des assertions** afin de formaliser la spécification.
- **Une assertion** est une expression de type booléen qui est censée être vrai lors de son évaluation. À défaut, l'assertion est violée.

➔ La présence des assertions facilite la localisation (et par conséquent la correction) des erreurs.

II. Classification des contrats au niveau des méthodes

Précondition: est une assertion qui doit être vérifiée **au moment de l'appel de la méthode**.

Postcondition: est une assertion qui doit être vérifiée **à la fin de l'exécution de la méthode**

Invariant: est une assertion qui doit rester **toujours vraie pendant toute l'exécution** de la méthode (ou d'un objet, dans le cas d'un invariant de classe).

Exemple : calcul de la racine carrée de x

- Précondition : $x \geq 0$
- Postconditions : résultat ≥ 0 et résultat^2 = x

II. Classification des contrats au niveau des méthodes

Exemple 1: cas du language C/C++:

```
#include<stdio.h>
#include<assert.h>
void main() {
int a,b;
int x;
printf("a=");
scanf("%d",&a);
printf("b=");
scanf("%d",&b);
if(a>b) {
   x=a;
   a=b;
   b=x;  }
/*affirmation a<=b*/
/* Pour rendre ce commentaire exécutable, on utilise assert /*
  assert(a<=b) ;

printf(" a=%d\t b=%d\n ",a, b) ;
```

exécution 1 :

a=4	**a=3**
b=3	**b=4**

exécution 2 :

a=3	**a=3**
b=3	**b=3**

Modification : Au lieu assert(a<=b) on suppose au même endroit assert(a<b)

(1)

a=4 et b=3

a=3 b=4

(2)

a=3 et b=3

assertion failed : a<b File:test.c line:20 abnormal program terminaison

II. Classification des contrats au niveau des méthodes

Exemple 2: cas du language Java

```
public class AssertTest1 {

public static void main( String args[] ) {

assert args.length >= 2 && args.length <= 4;

System.out.println("->" + args[3] );

}

}
```

→ Cette pré-condition, assure que que la méthode reçoit de 2 à 4 paramètres

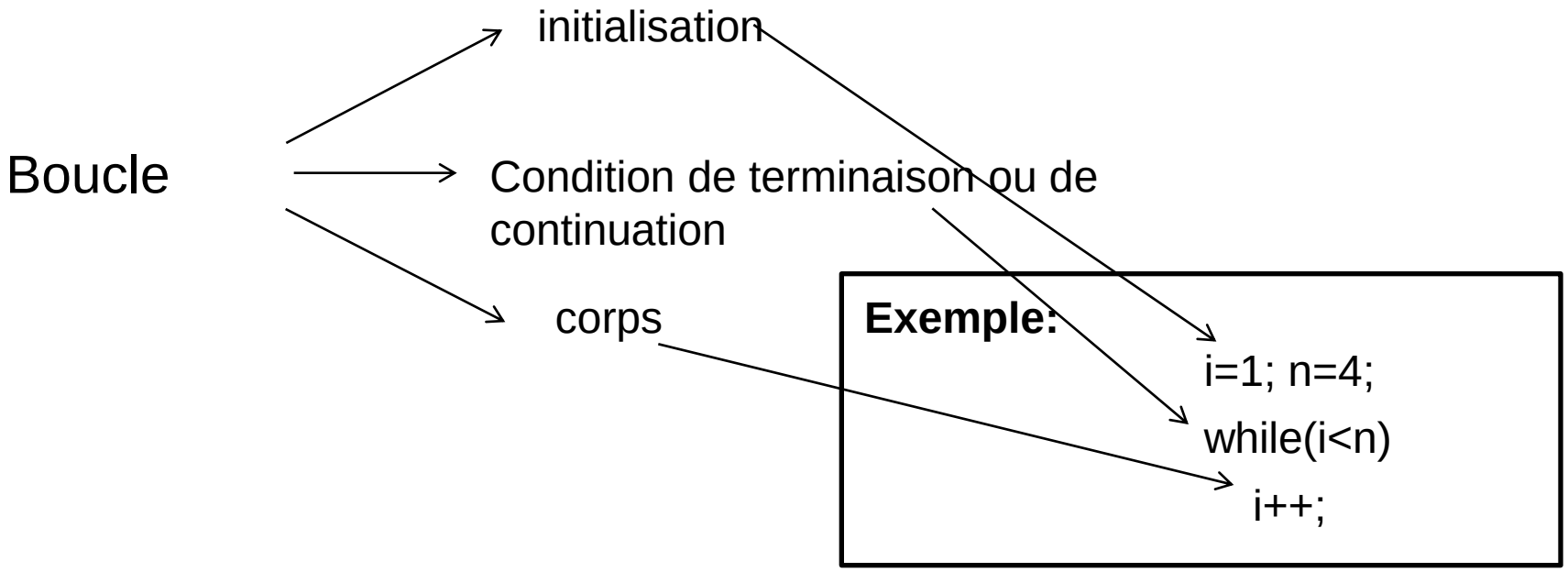
III. Classification des contrats au niveau des boucles

- Deux problèmes à résoudre :
  - Comment assurer la **terminaison** de la boucle
  ➔un variant de boucle

  - Comment assurer l'**exactitude** de la boucle: c.-à-d est ce que la boucle fait ou non le travail demandé
  ➔un invariant de boucle

III. Classification des contrats au niveau des boucles

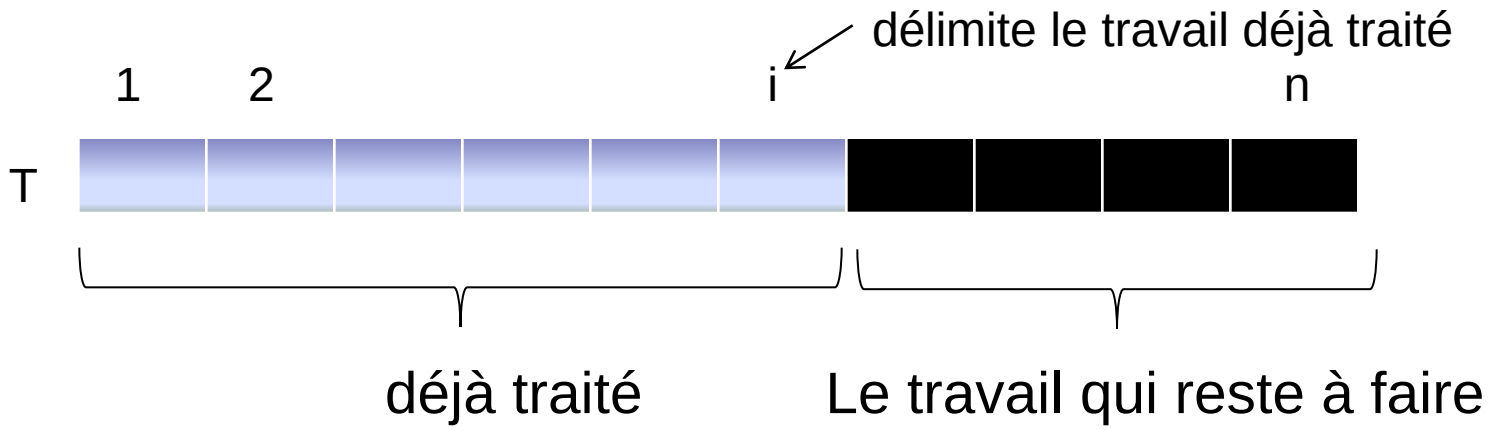
- Une boucle bien conçu comporte : un variant et un invariant



- **variant: n-i car (n-i) décroit et reste toujours >0**

III. Classification des contrats au niveau des boucles

Illustration: recherche de la valeur minimum dans un tableau T



Invariant: $\forall$ j $\in$ [1,i], min<=T[i]

III. Classification des contrats au niveau des boucles

```
#include<stdio.h>
#include<assert.h>
void main() {
    unsigned n=5, i;
    int T[n], min;
    for(i=0;i<n;i++)
                printf("donner une valeur");     scanf("%d",&T[i]);
     i=0;  min=T[i];
     for(i=0;i<n;i++) {
                if(min>T[i])
                        min=T[i];
                assert(min<=T[i ]); /*invariant de boucle */
                assert(n-i); /*variant de boucle car cette valeur décroit à
                                chaque itération sans qu'elle devienne négative */
               }
               printf("le minimun est %d ", min);
  }
```

IV. Exercices d'application

Exercice 1:

1) Ecrire un sous-programme (en utilisant le langage C) d'une recherche de la position de la valeur maximale d'un tableau d'entier.
2) Définir un variant et un invariant de boucle dans la solution proposée.
3) Améliorer la fonction par l'insertion des pré et post conditions

Exercice 2:

1) Ecrire un sous-programme (en utilisant le langage C) permettant d'effectuer une recherche dichotomique dans un tableau T supposé trié dans l'ordre croissant.
2) Définir un variant de boucle dans la solution proposée.
3) Définir un invariant de boucle dans la solution proposée.

IV. Exercices d'application (Rep. Question 2.1)

```
unsigned recherche_dichotomique (int info, int tab[], unsigned n) {
   int g,d ;/*sous tableau de recherche*/
   int m ;/*indice de l'élément au milieu*/
   int intervalle ; /*nombre d'élément de l'ensemble de recherche*/
/*initialisation*/
 g=0 ;
 d=n-1 ;
 do{
        m=(g+d)/2 ;/*division entière*/
         if(tab[m]==info) /*issue positive*/
                    return (1) ;
        /*soit déplacer g soit déplacer d jamais déplacer les deux à la fois*/
         if(info>T[m])
                     g=m+1;
        else
                     d=m-1;
} while(g<=d) ;
return 0 ; /*issue negative*/}
```

IV. Exercices d'application (Rep. Question 2.2)

```
unsigned recherche_dichotomique (int info, int tab[], unsigned n) {
   int g,d ;/*sous tableau de recherche*/
   int m ;/*indice de l'élément au milieu*/
/*initialisation*/
 g=0 ;
 d=n-1 ;
 do {
        m=(g+d)/2 ;/*division entière*/
        if(tab[m]==info) /*issue positive*/
                return (1) ;
 /*soit déplacer g soit déplacer d jamais déplacer les deux à la fois*/
       if(info>T[m])  g=m+1;
       else                d=m-1;
 } while(g<=d) ;
assert(d<g);  /*variant */
return 0; /*issue negative*/}
```

IV. Exercices d'application (Rep. Question 2.3)

```
unsigned recherche_dichotomique (int info, int tab[], unsigned n) {
   int g,d ;/*sous tableau de recherche*/
   int m ;/*indice de l'élément au milieu*/
   int intervalle ; /*nombre d'élément de l'ensemble de recherche*/
/*initialisation*/
 g=0 ;
 d=n-1 ;
 do {
        intervalle=d-g+1;
         m=(g+d)/2 ;/*division entière*/
         if(tab[m]==info) /*issue positive*/
                    return (1) ;
        /*soit déplacer g soit déplacer d jamais déplacer les deux à la fois*/
         if(info>T[m])   g=m+1;
        else                d=m-1;
        assert(intervalle>d-g+1); /*invariant  */
} while(g<=d) ;
return 0; /*issue negative*/}
```